Security Misconfiguration Issues

Elaheh Toulabi Nejad

Electrical and Computer Engineering Department
Isfahan University of Technology

What is Security Misconfiguration?

Security Misconfiguration is simply defined as failing to implement all the security controls for a server or web application, or implementing the security controls, but doing so with errors. What a company thought of as a safe environment actually has dangerous gaps or mistakes that leave the organization open to risk.

OWASP Ranking

OWASP Top 10 2017	2017	2013	2010	2007	2004
A1-Injection	X	X	X	X	X
A2-Broken Authentication	X	X	X	X	X
A3-Sensitive Data Exposure	X	X			
A4-XML External Entities (XXE)	X				
A5-Broken Access Control	X				
A6-Security Misconfiguration	X	X	X		X
A7-Cross-Site Scripting (XSS)	X	X	X	X	X
A8-Insecure Deserialization	X				
A9-Using Components with Known Vulnerabilities	X	X			
A10-Insufficient Logging & Monitoring	X				

Cause #1 : Inadvertant Use of Default Options

- Default options are always an easy target for hackers. It is very common that users often do not change their default password or do not delete default user ID.

- Some applications come with default port number as well.

Cause #2 : Excess Debug Options

- Revealing too much debugging information is a very common misconfiguration problem. This usually does not result directly to exploitation of a system.
- Attackers can collect extra information, such as the internal working of an application and version numbers.
- Attackers can use this excessive debugging information to craft SQL to perform a SQL injection attack. Also, when applications fail to perform an action, they can leak sensitive information.

Cause #3 : Improper Policy or Role Configuration

- Role misconfiguration is another leading cause of web application misconfiguration.
- This causes groups or roles to access settings or records that were not intended for them. There are many reasons for role misconfigurations.
- Complex business roles and polices can be blamed for role misconfiguration. Example: allowing admin staff to view human resource data.

Cause #4 : Human Mistakes ☺

- Human mistakes are frequent and unavoidable and can account for up to 43% of all system failures.
- Operator error is the main reason for downtime for large websites, such as Google, MSN, and Yahoo. Proper interfaces and good design can drastically reduce operator mistakes.
- Poorly designed application interfaces and too tight restriction may force user to attempt or bypass security then they need to accomplish a goal.

Where?

Security misconfiguration can happen at any level of an application stack, including:

- the network services
- platform
- web server
- application server
- database
- frameworks
- custom code
- pre-installed virtual machines, containers, or storage

Impacts

Such flaws frequently give attackers unauthorized access to some system data or functionality. Occasionally, such flaws result in a complete system compromise.

The business impact depends on the protection needs of the application and data. Therefore, this opens the door to impact *Confidentiality, Integrity and Availability,* depending on the context.

Is the Application Vulnerable? (1)

The application might be vulnerable if the application is:

- Missing appropriate security hardening across any part of the application stack, or improperly configured permissions on cloud services.
- Unnecessary features are enabled or installed (e.g. unnecessary ports, services, pages, accounts, or privileges).
- Default accounts and their passwords still enabled and unchanged.
- Error handling reveals stack traces or other overly informative error messages to users.

Is the Application Vulnerable? (2)

- For upgraded systems, latest security features are disabled or not configured securely.
- The security settings in the application servers, application frameworks (e.g. Struts, Spring, ASP.NET), libraries, databases, etc. not set to secure values.
- The server does not send security headers or directives or they are not set to secure values.
- The software is out of date or vulnerable (A9:2017-Using Components with Known Vulnerabilities).

How to Prevent (1)

Basic Policies :

- Do not use default credentials.
- Avoid default installations.
- Maintain consistency of configuration between versions.
- Restrict default configuration options.
- Avoid default port numbers.
- Restrict roles and privileges.
- Centralize configuration as much as possible.
- Scans and audits.
- Strong encryption

How to Prevent (2)

Secure Coding and Code Review:

- Design application functionality with security in mind.
- Extends development time.
- Practice defensive coding.
- Review codes to ensure security properties.
- Stay up to date with coding standards compliance.
- Consistent design and implementation.
- Finding security issues/bugs early

How to Prevent (3)

Secure Configuration and Testing:

- Attention to detail .
- Each component should be checked and verified.
- Turn off unsafe features.
- Remove default accounts and expire default passwords.
- Stress testing.
- Penetration testing.
- Both automatic and manual testing.

How to Prevent (4)

Patching and Education:

- Keep applications up to date.
- Apply vendor patches on time.
- Apply critical security and vulnerability regularly.
- Educate developers, administrators, and testers.
- Subscribe to vendor’s security alert.

Demo #1

Demo #2